

Advanced Machine Learning

T.A Merit Samir



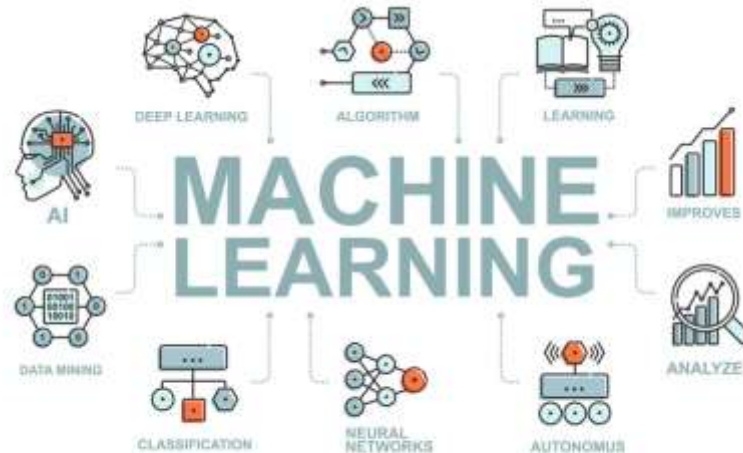
Lab 2

1. Machine Learning
2. Types of Machine Learning
3. Classification
4. KNN (Key Nearest Neighbors)
5. Naive Bayes



Machine learning

Machine learning (ML) is a subset of artificial intelligence (AI) that allows systems to learn from data and improve their performance over time without being explicitly programmed. It's based on algorithms that can identify patterns, make decisions, and predict outcomes based on data.



Types of Machine Learning:

1.Supervised Learning: Involves training a model on labeled data, where the correct output is known. It's used for tasks like classification (e.g., spam detection) and regression (e.g., predicting house prices).

2.Unsupervised Learning: Involves working with data that is not labeled. The goal is to discover hidden patterns or intrinsic structures within the data. Common tasks include clustering (e.g., customer segmentation) and association (e.g., market basket analysis).

3.Reinforcement Learning: The model learns by interacting with an environment and receiving feedback through rewards or penalties. It's often used in robotics, gaming, and autonomous systems.



Classification

Classification is a type of supervised machine learning where the goal is to categorize or classify data points into predefined classes or labels.

It is used when the output variable is categorical (e.g., "yes" or "no," "spam" or "not spam," "dog" or "cat"). Here's a detailed look at classification:

Types of Classification

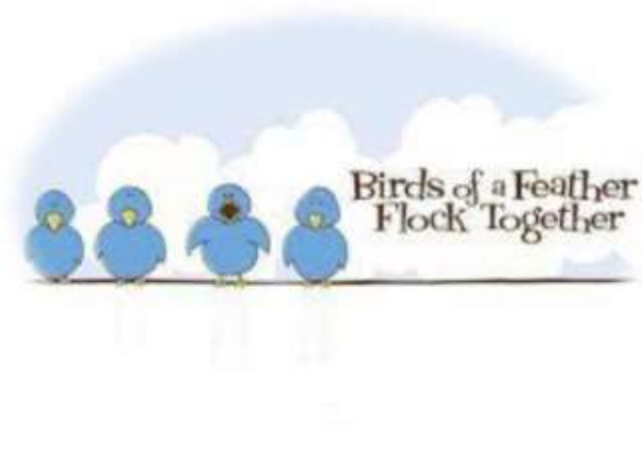
- **Binary Classification:** Involves two classes (e.g., spam detection where emails are classified as either "spam" or "not spam").
- **Multiclass Classification:** Involves more than two classes (e.g., classifying types of animals such as "dog," "cat," or "horse").
- **Multilabel Classification:** Each data point can belong to multiple classes simultaneously (e.g., an image may contain both a "dog" and a "cat").



K-Nearest Neighbors (KNN)

KNN algorithm assumes that similar things exist in proximity.
In other words, similar things are near to each other.

- Notice in the image that most of the time, similar data points are close to each other.
- KNN captures the idea of similarity (sometimes called distance, proximity, or closeness) with some mathematics we might have learned in our childhood— calculating the distance between points on a graph.



K-Nearest Neighbors (KNN)

- supervised classification/regression algorithm.
- An object is classified by a plurality vote of its neighbors, with the object being assigned to the class most common among its k nearest neighbors (k is a positive integer, typically small).
- If $k = 1$, then the object is simply assigned to the class of that single nearest neighbor.

K-Nearest Neighbors: Distance Metric

L1 (Manhattan) distance

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$

L2 (Euclidean) distance

$$d_1(I_1, I_2) = \left(\sum_p (I_1^p - I_2^p)^2 \right)^{\frac{1}{2}}$$

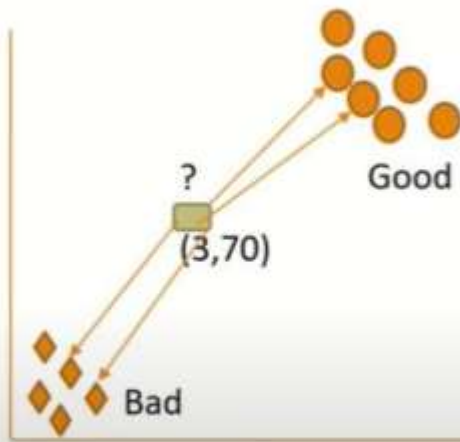


K = 1



K-Nearest Neighbors Cont.

K=4



Name	Cigarettes	Weight	Heart Attack	Distance	
A	7	70	Bad	$\sqrt{(3-7)^2 + (70-70)^2}$ = 4	1
B	7	40	Bad	$\sqrt{(3-7)^2 + (70-40)^2}$ = 30.27	4
C	3	40	Good	$\sqrt{(3-3)^2 + (70-40)^2}$ = 30.00	2
D	1	40	Good	$\sqrt{(3-1)^2 + (70-40)^2}$ = 30.07	3
E	3	70	Bad		



K-Nearest Neighbors Cont.

KNN pros and cons

Pros

1. No Training Period: KNN is called Lazy Learner (Instance based learning).
2. new data can be added seamlessly which will not impact the accuracy of the algorithm.
3. KNN is very easy to implement, There are only two parameters required to implement KNN i.e. the value of K and the distance function (e.g. Euclidean or Manhattan etc.)

Cons

1. Does not work well with large dataset:
2. Does not work well with high dimensions
3. Need feature scaling: We need to do feature scaling (standardization and normalization)
4. Sensitive to noisy data, missing values.



K-Nearest Neighbors Code Example

Read
Social_Network_Ads
dataset

Social_Network_Ads.csv X ...

1 to 10 of 400 entries Filter

Age	EstimatedSalary	Purchased
19	19000	0
35	20000	0
26	43000	0
27	57000	0
19	76000	0
27	58000	0
27	84000	0
32	150000	1
25	33000	0
35	65000	0



KNN Cont.

```
✓ [1] import numpy as np  
1s      import matplotlib.pyplot as plt  
      import pandas as pd
```

✓ Importing the dataset

```
✓ [2] dataset = pd.read_csv('Social_Network_Ads.csv')  
0s      X = dataset.iloc[:, :-1].values  
      y = dataset.iloc[:, -1].values
```

✓ Splitting the dataset into the Training set and Test set

```
✓ [3] from sklearn.model_selection import train_test_split  
3s      X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25, random_state = 0)
```



KNN Cont.

✓ Feature Scaling

```
✓ [8] from sklearn.preprocessing import StandardScaler  
s    sc = StandardScaler()  
      X_train = sc.fit_transform(X_train)  
      X_test = sc.transform(X_test)
```

```
✓ [9] print(X_train)
```



KNN Cont.

✓ Training the K-NN model on the Training set

```
[11] from sklearn.neighbors import KNeighborsClassifier  
      classifier = KNeighborsClassifier(n_neighbors = 5, metric = 'minkowski', p = 2)  
      classifier.fit(X_train, y_train)
```



▼ KNeighborsClassifier ⓘ ⓘ
KNeighborsClassifier()

Instantiate the KNN classifier object with specific parameters:

- **n_neighbors** = 5: This specifies that the algorithm will use the 5 nearest neighbors to make a prediction.
- **metric** = 'minkowski': This is the distance metric used to calculate the distance between data points. The Minkowski distance is a generalization of both the Euclidean and Manhattan distances.
- **p** = 2: The parameter p defines the order of the Minkowski distance. When p = 2, the distance metric becomes Euclidean distance. If you set p = 1, it becomes Manhattan distance.



Faculty of Computer Science & Engineering - New Mansoura University


$$\begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 1 \\ 0 & 0 \\ 0 & 0 \\ 1 & 1 \\ 0 & 0 \\ 1 & 1 \\ 1 & 1 \\ 0 & 0 \\ 0 & 0 \\ 1 & 1 \\ 0 & 0 \\ 0 & 0 \\ 1 & 1 \\ 0 & 0 \\ 1 & 1 \\ 0 & 0 \\ 1 & 1 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

1.1.1

KNN Cont.

✓ Making the Confusion Matrix

```
[14] from sklearn.metrics import confusion_matrix, accuracy_score  
cm = confusion_matrix(y_test, y_pred)  
print(cm)  
accuracy_score(y_test, y_pred)
```

⇒ $\begin{bmatrix} 64 & 4 \\ 3 & 29 \end{bmatrix}$ $\begin{bmatrix} \text{True Negative (TN)} & \text{False Positive (FP)} \\ \text{False Negative (FN)} & \text{True Positive (TP)} \end{bmatrix}$
0.93



KNN Cont.

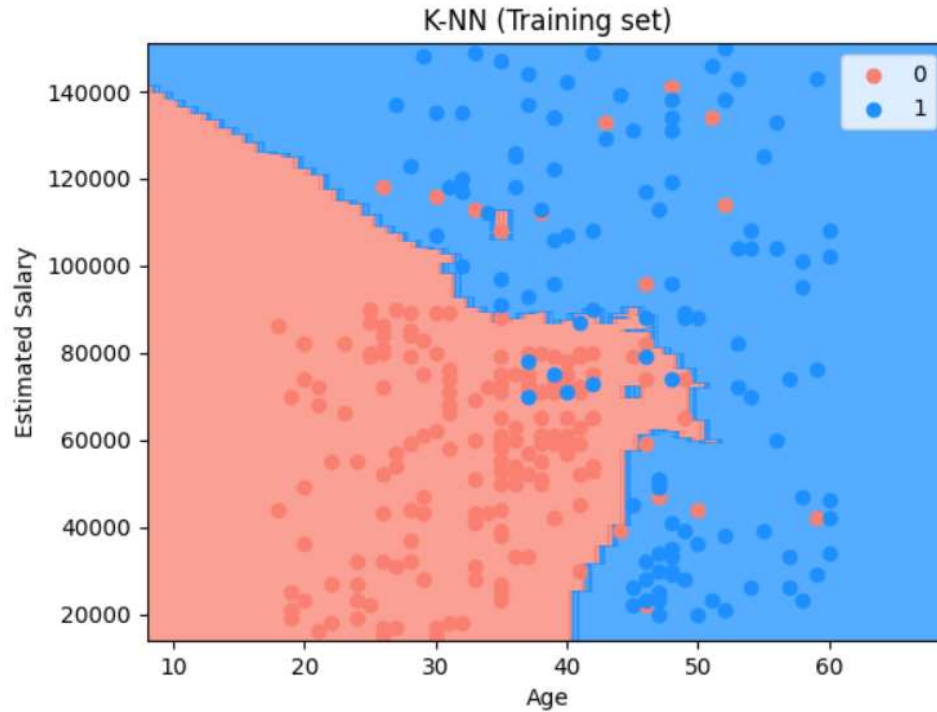
✓ Visualising the Training set results



```
from matplotlib.colors import ListedColormap
X_set, y_set = sc.inverse_transform(X_train, y_train)
X1, X2 = np.meshgrid(np.arange(start = X_set[:, 0].min() - 10, stop = X_set[:, 0].max() + 10, step = 1),
                     np.arange(start = X_set[:, 1].min() - 1000, stop = X_set[:, 1].max() + 1000, step = 1))
plt.contourf(X1, X2, classifier.predict(sc.transform(np.array([X1.ravel(), X2.ravel()])).T).reshape(X1.shape),
             alpha = 0.75, cmap = ListedColormap(('salmon', 'dodgerblue')))
plt.xlim(X1.min(), X1.max())
plt.ylim(X2.min(), X2.max())
for i, j in enumerate(np.unique(y_set)):
    plt.scatter(X_set[y_set == j, 0], X_set[y_set == j, 1], c = ListedColormap(('salmon', 'dodgerblue'))(i), label = j)
plt.title('K-NN (Training set)')
plt.xlabel('Age')
plt.ylabel('Estimated Salary')
plt.legend()
plt.show()
```



KNN Cont.



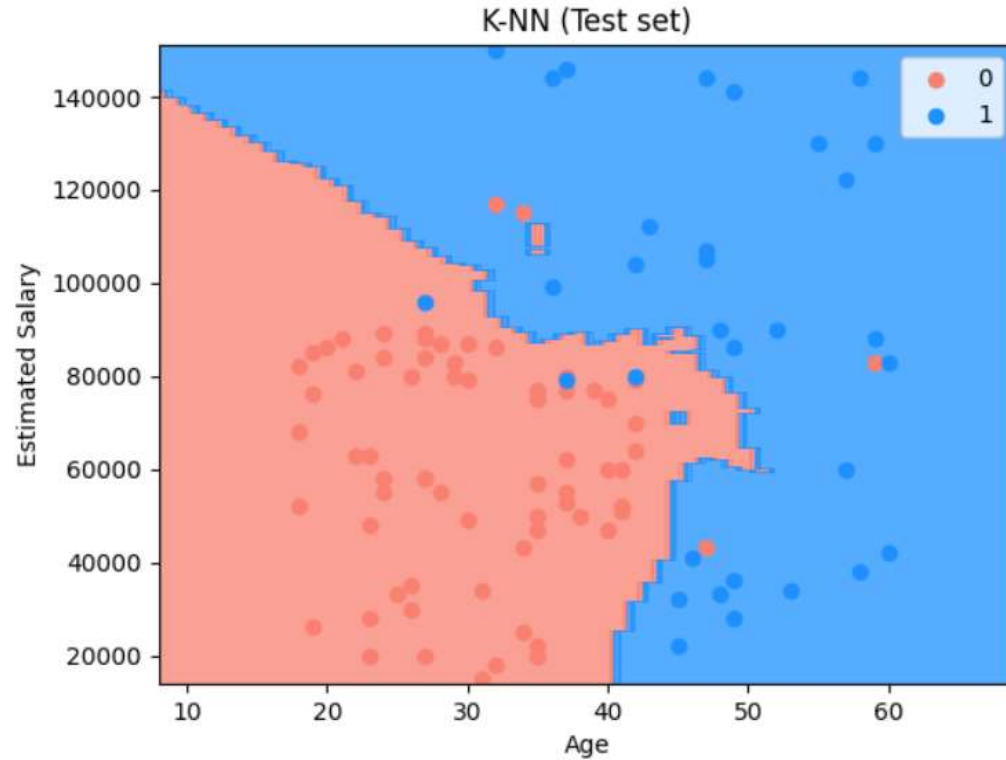
KNN Cont.

✓ Visualising the Test set results

```
[17] from matplotlib.colors import ListedColormap
X_set, y_set = sc.inverse_transform(X_test), y_test
X1, X2 = np.meshgrid(np.arange(start = X_set[:, 0].min() - 10, stop = X_set[:, 0].max() + 10, step = 1),
                     np.arange(start = X_set[:, 1].min() - 1000, stop = X_set[:, 1].max() + 1000, step = 1))
plt.contourf(X1, X2, classifier.predict(sc.transform(np.array([X1.ravel(), X2.ravel()]).T)).reshape(X1.shape),
             alpha = 0.75, cmap = ListedColormap(('salmon', 'dodgerblue')))
plt.xlim(X1.min(), X1.max())
plt.ylim(X2.min(), X2.max())
for i, j in enumerate(np.unique(y_set)):
    plt.scatter(X_set[y_set == j, 0], X_set[y_set == j, 1], c = ListedColormap(('salmon', 'dodgerblue'))(i), label = j)
plt.title('K-NN (Test set)')
plt.xlabel('Age')
plt.ylabel('Estimated Salary')
plt.legend()
plt.show()
```



KNN Cont.



Naive Bayes

Naive Bayes is a simple yet powerful classification algorithm based on Bayes' Theorem with an assumption of **conditional independence** between features. Despite its simplicity, it is highly effective for tasks like text classification, spam detection, and sentiment analysis.

Naive Bayes is based on **Bayes' Theorem**, which describes the probability of an event based on prior knowledge of related events. The theorem is expressed as:

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$



Naive Bayes

Play Tennis Example

PlayTennis: training examples

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No



Naive Bayes

Calculating Probabilities

• Learning Phase

Outlook	Play=Yes	Play=No	Temperature	Play=Yes	Play=No
Sunny	2/9	3/5	Hot	2/9	2/5
Overcast	4/9	0/5	Mild	4/9	2/5
Rain	3/9	2/5	Cool	3/9	1/5

Humidity	Play=Yes	Play=No	Wind	Play=Yes	Play=No
High	3/9	4/5	Strong	3/9	3/5
Normal	6/9	1/5	Weak	6/9	2/5

$$P(\text{Play=Yes}) = 9/14 \quad P(\text{Play=No}) = 5/14$$



Naive Bayes

Test Phase

–Given a new instance, predict its label

$\mathbf{x}=(\text{Outlook}=\text{Sunny}, \text{Temperature}=\text{Cool}, \text{Humidity}=\text{High}, \text{Wind}=\text{Strong})$

–Look up tables achieved in the learning phase $\mathcal{I}$

Outlook	Play=Yes	Play=No
Sunny	2/9	3/5
Overcast	4/9	0/5
Rain	3/9	2/5

Temperature	Play=Yes	Play=No
Hot	2/9	2/5
Mild	4/9	2/5
Cool	3/9	1/5

Humidity	Play=Yes	Play=No
High	3/9	4/5
Normal	6/9	1/5

Wind	Play=Yes	Play=No
Strong	3/9	3/5
Weak	6/9	2/5

$$P(\text{Play}=\text{Yes}) = 9/14 \quad P(\text{Play}=\text{No}) = 5/14$$

$$P(\text{Outlook}=\text{Sunny}|\text{Play}=\text{Yes}) = 2/9$$

$$P(\text{Outlook}=\text{Sunny}|\text{Play}=\text{No}) = 3/5$$

$$P(\text{Temperature}=\text{Cool}|\text{Play}=\text{Yes}) = 3/9$$

$$P(\text{Temperature}=\text{Cool}|\text{Play}=\text{No}) = 1/5$$

$$P(\text{Humidity}=\text{High}|\text{Play}=\text{Yes}) = 3/9$$

$$P(\text{Humidity}=\text{High}|\text{Play}=\text{No}) = 4/5$$

$$P(\text{Wind}=\text{Strong}|\text{Play}=\text{Yes}) = 3/9$$

$$P(\text{Wind}=\text{Strong}|\text{Play}=\text{No}) = 3/5$$

$$P(\text{Play}=\text{Yes}) = 9/14$$

$$P(\text{Play}=\text{No}) = 5/14$$

$$P(\text{Yes}|\mathbf{x}') \approx [P(\text{Sunny}|\text{Yes})P(\text{Cool}|\text{Yes})P(\text{High}|\text{Yes})P(\text{Strong}|\text{Yes})]P(\text{Play}=\text{Yes}) = 0.0053$$

$$P(\text{No}|\mathbf{x}') \approx [P(\text{Sunny}|\text{No})P(\text{Cool}|\text{No})P(\text{High}|\text{No})P(\text{Strong}|\text{No})]P(\text{Play}=\text{No}) = 0.0206$$

Given the fact $P(\text{Yes}|\mathbf{x}') < P(\text{No}|\mathbf{x}')$, we label $\mathbf{x}'$ to be "No".



Naive Bayes

Pros and Cons

Advantages of Naive Bayes

1. This algorithm works very fast and can easily predict the class of a test dataset.
2. You can use it to solve multi-class prediction problems as it's quite useful with them.
3. Naive Bayes classifier performs better than other models with less training data if the assumption of independence of features holds.

Disadvantages of Naive Bayes

1. If your test data set has a categorical variable of a category that wasn't present in the training data set, the Naive Bayes model will assign it zero probability and won't be able to make any predictions in this regard.
2. It assumes that all the features are independent. While it might sound great in theory, in real life, you'll hardly find a set of independent features.



Naive Bayes Example

Read
Social_Network_Ads
dataset

Social_Network_Ads.csv X ...

1 to 10 of 400 entries Filter

Age	EstimatedSalary	Purchased
19	19000	0
35	20000	0
26	43000	0
27	57000	0
19	76000	0
27	58000	0
27	84000	0
32	150000	1
25	33000	0
35	65000	0



Naive Bayes

```
✓ [1] import numpy as np  
1s      import matplotlib.pyplot as plt  
      import pandas as pd
```

✓ Importing the dataset

```
✓ [2] dataset = pd.read_csv('Social_Network_Ads.csv')  
0s      X = dataset.iloc[:, :-1].values  
      y = dataset.iloc[:, -1].values
```

✓ Splitting the dataset into the Training set and Test set

```
✓ [3] from sklearn.model_selection import train_test_split  
3s      X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25, random_state = 0)
```



Naive Bayes

✓ Feature Scaling

```
✓ [8] from sklearn.preprocessing import StandardScaler  
s    sc = StandardScaler()  
      X_train = sc.fit_transform(X_train)  
      X_test = sc.transform(X_test)
```

```
✓ [9] print(X_train)  
s
```



Naive Bayes

✓ Training the Naive Bayes model on the Training set

✓
0s



```
from sklearn.naive_bayes import GaussianNB  
classifier = GaussianNB()  
classifier.fit(X_train, y_train)
```



▼ GaussianNB ⓘ ⓘ
GaussianNB()

Imports the GaussianNB class from the sklearn.naive_bayes module.

The Gaussian Naive Bayes algorithm is used when the features (input variables) follow a Gaussian (normal) distribution.

This creates an instance of the GaussianNB classifier.

There are no additional hyperparameters to set here for the basic Gaussian Naive Bayes model.



Naive Bayes

✓ Predicting a new result

```
[12] print(classifier.predict(sc.transform([[30,87000]])))
```

⇒ [0]

✓ Predicting the Test set results

```
[13] y_pred = classifier.predict(X_test)
      print(np.concatenate((y_pred.reshape(len(y_pred),1), y_test.reshape(len(y_test),1)),1))
```

[0 0]
[0 0]
[1 1]
[0 0]
[0 0]
[1 1]
[0 0]
[1 1]
[1 1]
[0 0]
[0 0]
[1 0]
[1 1]
[1 1]
[0 0]
[0 0]
[1 1]
[0 0]
[0 0]
[1 1]
[0 0]
[1 1]
[0 0]
[0 0]
[0 0]
[0 0]
[0 0]
[0 0]
[0 0]
[0 0]
[0 0]



Naive Bayes

Making the Confusion Matrix

```
from sklearn.metrics import confusion_matrix, accuracy_score  
cm = confusion_matrix(y_test, y_pred)  
print(cm)  
accuracy_score(y_test, y_pred)
```

[14]

```
... [[65  3]  
     [ 7 25]]
```

```
... 0.9
```



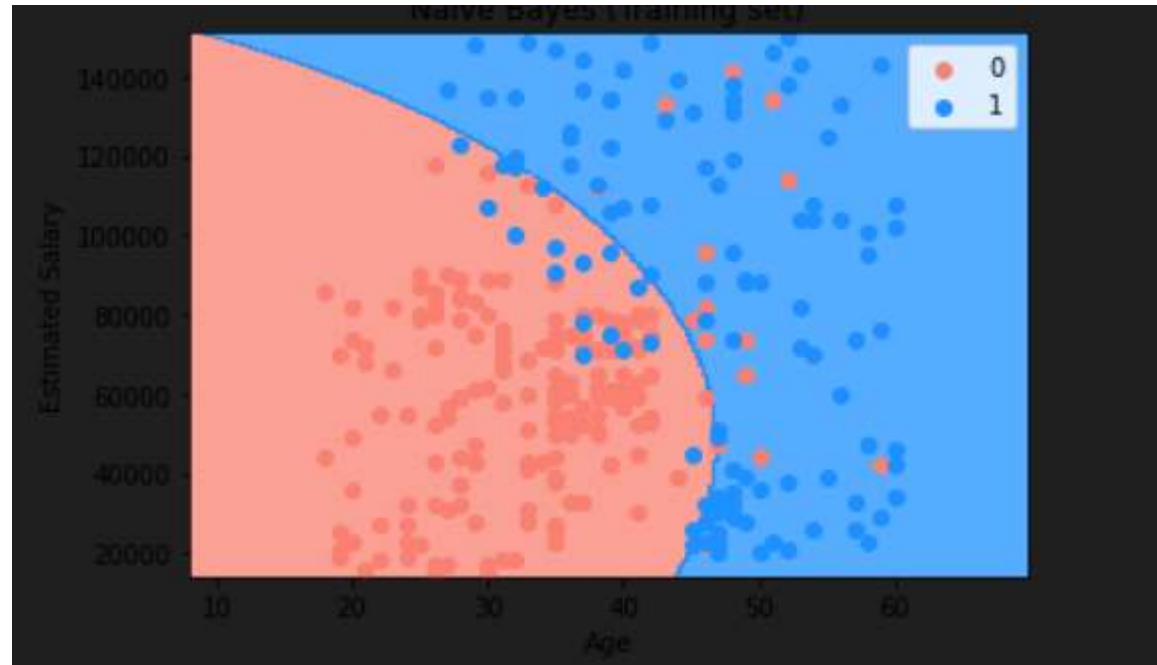
Naive Bayes

Visualising the Training set results

```
from matplotlib.colors import ListedColormap
X_set, y_set = sc.inverse_transform(X_train), y_train
X1, X2 = np.meshgrid(np.arange(start = X_set[:, 0].min() - 10, stop = X_set[:, 0].max() + 10, step = 0.25),
                      np.arange(start = X_set[:, 1].min() - 1000, stop = X_set[:, 1].max() + 1000, step = 0.25))
plt.contourf(X1, X2, classifier.predict(sc.transform(np.array([X1.ravel(), X2.ravel()]).T)).reshape(X1.shape),
             alpha = 0.75, cmap = ListedColormap(('salmon', 'dodgerblue')))
plt.xlim(X1.min(), X1.max())
plt.ylim(X2.min(), X2.max())
for i, j in enumerate(np.unique(y_set)):
    plt.scatter(X_set[y_set == j, 0], X_set[y_set == j, 1], c = ListedColormap(('salmon', 'dodgerblue'))(i), label = j)
plt.title('Naive Bayes (Training set)')
plt.xlabel('Age')
plt.ylabel('Estimated Salary')
plt.legend()
plt.show()
```



Naive Bayes



Naive Bayes

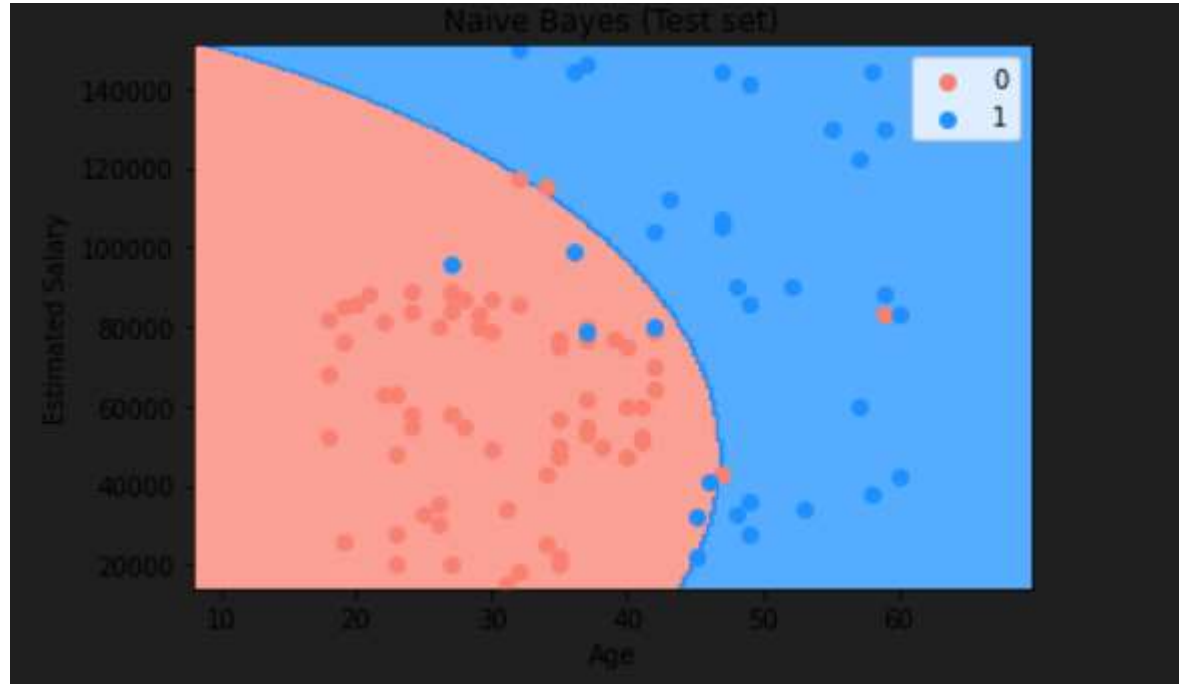
Visualising the Test set results

```
from matplotlib.colors import ListedColormap
X_set, y_set = sc.inverse_transform(X_test), y_test
X1, X2 = np.meshgrid(np.arange(start = X_set[:, 0].min() - 10, stop = X_set[:, 0].max() + 10, step = 0.25),
                     np.arange(start = X_set[:, 1].min() - 1000, stop = X_set[:, 1].max() + 1000, step = 0.25))
plt.contourf(X1, X2, classifier.predict(sc.transform(np.array([X1.ravel(), X2.ravel()]).T)).reshape(X1.shape),
             alpha = 0.75, cmap = ListedColormap(('salmon', 'dodgerblue')))
plt.xlim(X1.min(), X1.max())
plt.ylim(X2.min(), X2.max())
for i, j in enumerate(np.unique(y_set)):
    plt.scatter(X_set[y_set == j, 0], X_set[y_set == j, 1], c = ListedColormap(('salmon', 'dodgerblue'))(i), label = j)
plt.title('Naive Bayes (Test set)')
plt.xlabel('Age')
plt.ylabel('Estimated Salary')
plt.legend()
plt.show()
```

[16]



Naive Bayes





merit.samir@nmu.edu.eg



Thank you

